

Film Festival Event

U25161360, U24583741, U25087984

Task 1

1.1

French monochromatic Silencieux film festival with outside zones to meet your like minded moving picture arteurs, Programme Areas to hear talks from the arteurs behind your favourite works and Venues where you can experience the latest and newest films.

Event Groups:

Zone

Venue

ProgrammeArea

Event Leafs:

Tent

Food Truck

Screen (Both Subject and Observer, causes Venue to close)

Popcorn Stand

Bathroom

Medical

Stage (Both Subject and Observer, causes Venue to close)

1.2

Composite: ProgrammeArea, Zone, Venue

Component: EventComponent

Leaf: Medical, FoodTruck, Stage, Screen, Tent, Medical, PopCorn, Bathroom

Subject: Subject

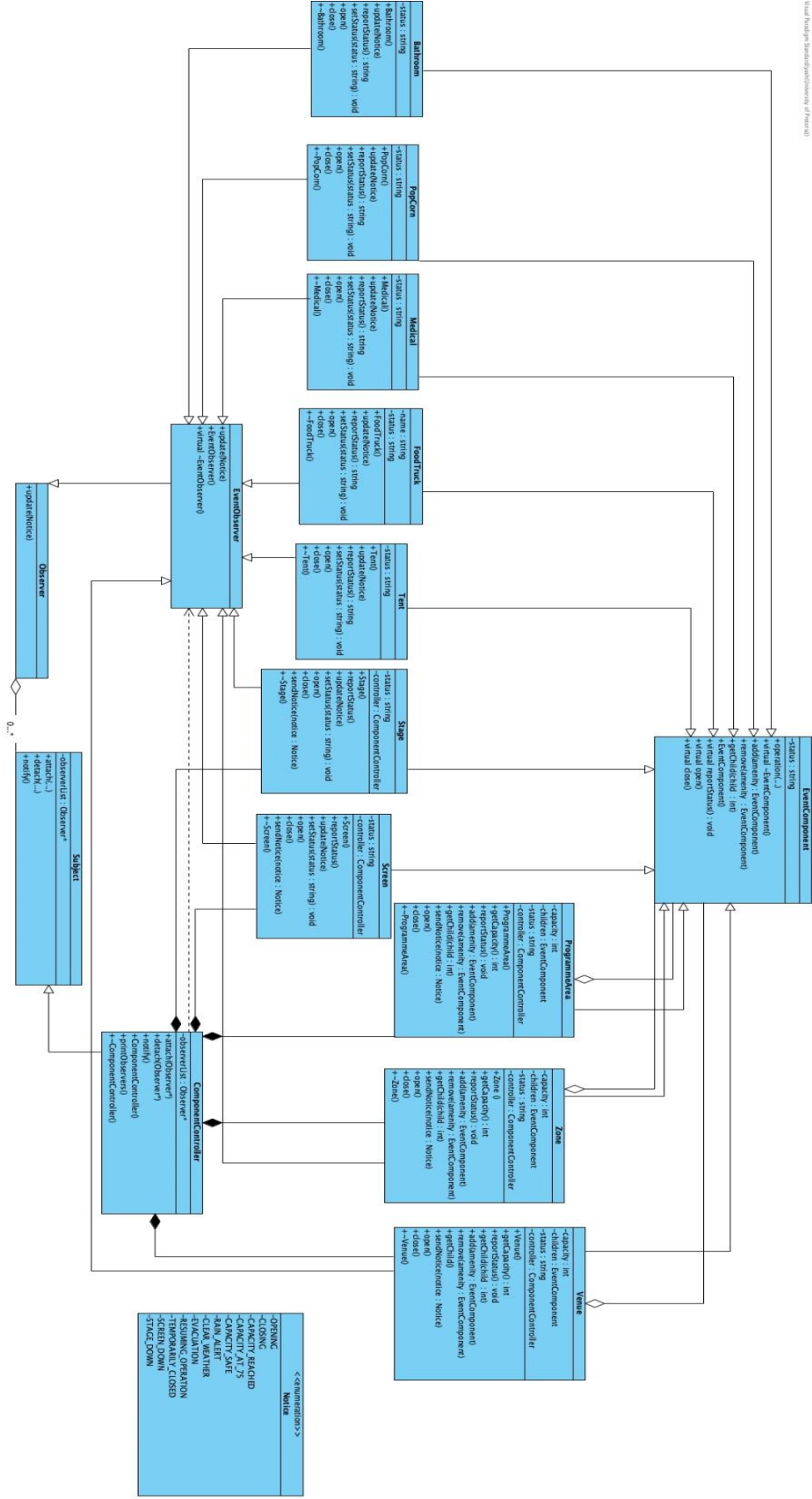
Observer: Observer

ConcreteObserver: Medical, FoodTruck, Stage, Screen, Tent, PopCorn, Bathroom

ConcreteSubject: Screen, Stage, ProgrammeArea, Zone, Venue

The Stage and Screen classes play dual rolls as if they do not work or are closed the event composite hosting them may have to change its behaviour

1.3



1.4a) Our composite is a genuine part-whole tree because the composites and the leaves inherit from the same component, the composites like zones and venues hold references to other components in the tree, there is no graph cycling and every part child belongs to one whole parent

1.4b) An observer is required because we want to maintain Open/Closed Principle and be able to add new types of Notices without having to change the code in to if/else statements that directly hard call

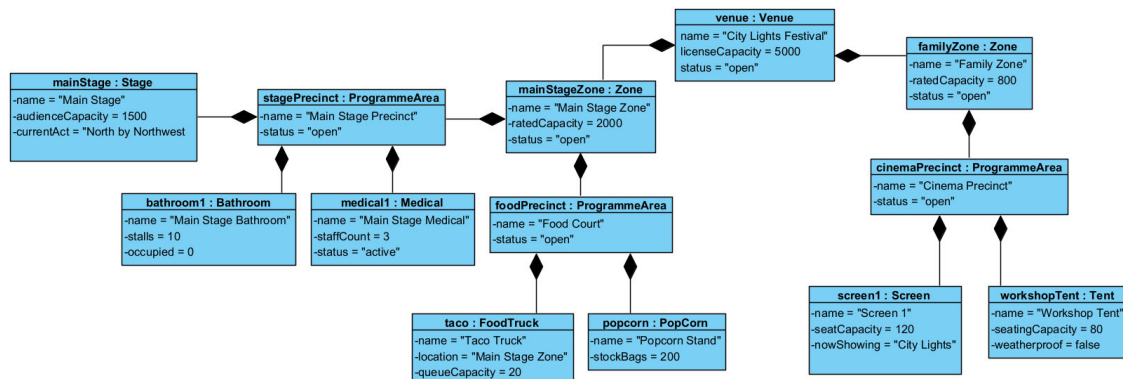
1.4c) Composites own event components

1.4d) The Subject owns the observers as its children but does not destruct them, it just changes their values based on its notices

1.4e) Multi-inheritance is allowed in C++, a class inheriting from multiple classes is not a misuse because each pattern performs separate roles.

Task 2

2.3



2.4

- Every Composite (ProgrammeArea, Zone, Venue) owns whatever is currently in its children vector. Ownership is exclusive: a component only ever sits in one parent's vector at a time.
- remove() only detaches a pointer from that vector; it does not delete it. This lets the same operation support two different needs:
- Permanent removal: parent->remove(child) followed by an explicit delete child by the caller.
- Transfer: oldParent->remove(child) followed by newParent->add(child). The object is briefly unowned but never double-owned, so there is no path to a double delete.
- This exclusive-ownership rule is what guarantees the root's destructor releases the whole subtree exactly once.

- Each composite's destructor loops over its own children and deletes whatever remains.
- A single delete on the root Venue recursively triggers Zone's destructor on each owned Zone, which triggers ProgrammeArea's destructor on each owned ProgrammeArea, down to the Leaves.
- Every object is reachable through exactly one destructor chain, so nothing is deleted twice and nothing is skipped.
- This was verified directly: SampleEvent.cpp builds the 13-object, four-level tree, calls delete on the root once, and the program runs clean under AddressSanitizer and LeakSanitizer with zero leaks or invalid frees.
- This chain only works because EventComponent's destructor is virtual.
- Deleting through a base EventComponent* still invokes the correct most-derived destructor first (e.g. Venue, then Zone, then EventComponent).
- Without the virtual destructor, derived cleanup would be silently skipped and anything that class owned would leak.
- The same transfer rule matters again in Task 3.
- Moving a unit between composites at runtime must also update its Observer registrations.
- Ownership and observation are separate relationships, and both need to stay consistent during a transfer.

Task 3

3.5 Push: Observer::update(Notice n) takes the Notice by value; ComponentController::sendNotice(n) sets it and calls notify(), which loops observers calling update(currentNotice). So the exact payload transferred is a single Notice enum value (e.g. RAIN_ALERT) — nothing else, and each composite in the cascade just forwards that same value on.

Pros:

- Notice is already a complete, self-describing message — no extra state to fetch
- Keeps update() a one-line switch(n)
- Observer never needs a back-pointer to the Subject
- Fits the cascade in 5.2: each level just re-broadcasts the value it received

Cons:

- Only cheap because the payload is small and fixed-shape
- If a notice ever needed extra data (e.g. *how much* rain), Notice would need to become a struct, touching every update() override
- Pull avoids that by calling back subject->getState(), at the cost of coupling Observer to the concrete Subject

Task 4

EVENT RULES

1. Screen goes down then a venue must close and all children must close and schedule must change
2. Stage goes down then a program area must close
3. During bad weather, zones must close and medical stays open
4. When clear weather open everything in a zone
5. When temporarily closed close everything but popcorn and medical

Runtime changes

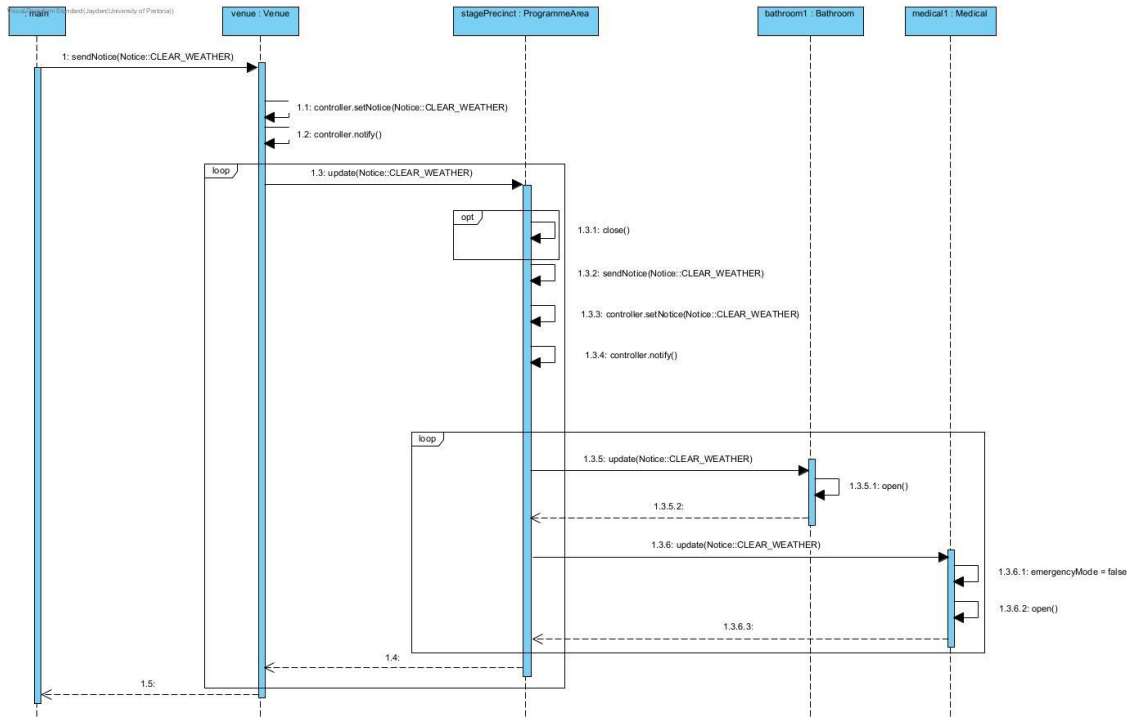
Transfer a medical leaf to another composite, pass composite and leaf

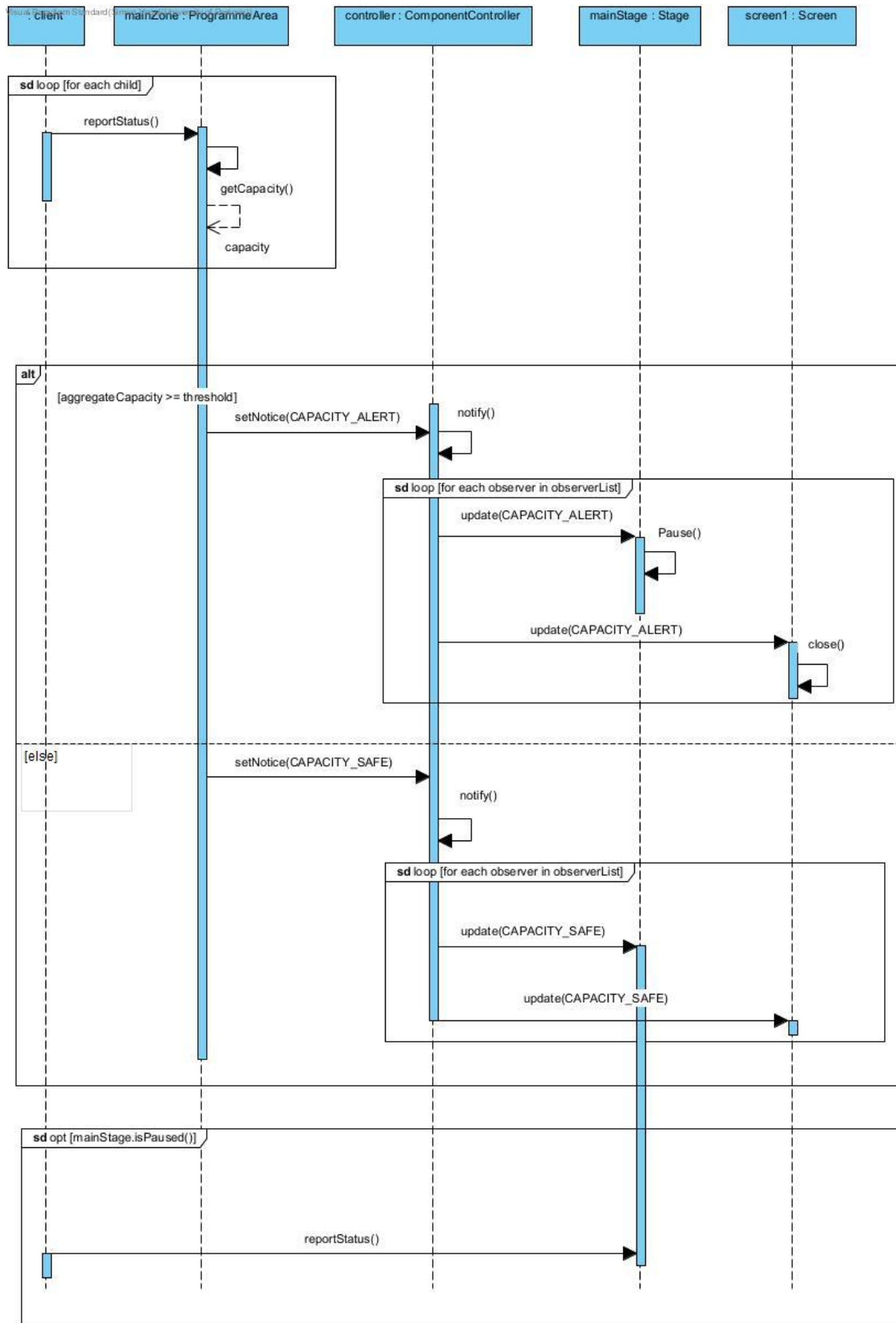
CONDITIONAL

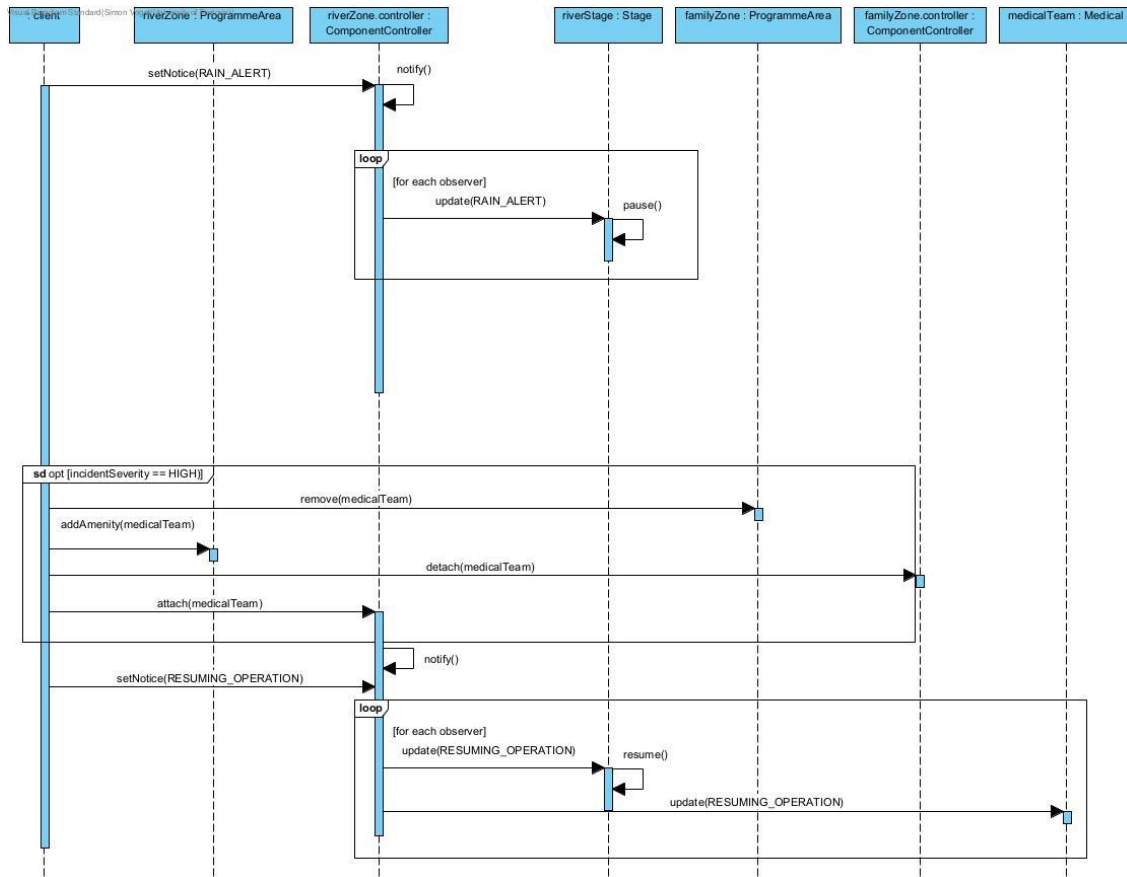
If capacity reached venue/PA or zone must close down

Task 5









DOXYGEN

EventFlow

Live Event Coordination Engine - COS214 Prac 3

Main Page
Classes
Files

EventFlow
Classes
Files

EventFlow Documentation

Generated by **doxygen** 1.9.1

GitHub

<https://github.com/yashGhela/COS214Prac3/tree/main>

Using GitHub streamlined working as a team like a charm. Instead of having to email files or upload them to google drive, we could easily push and pull code to each other. The tasks were split up evenly among team members, and each member had responsibility for one of the coding tasks(task 2, 3 or 4). These 3 tasks required completion of the preceding or preceding task to work, we each constantly needed updated code from our fellow members on the fly. This would not have been possible(or at least extremely cumbersome) without GitHub. We all worked in separate branches, this way our individual work wouldn't destroy the project as a whole. Then when our individual tasks were completed or someone needed the updated code we were working on, we would merge our branches to the main.

Team Contributions

Yash:

- I completed task 1. We first had a discord call to brainstorm the answers and structure needed for task 1 so I could go and formalise it.
- I completed task 4. Using github to coordinate the work.
- I also contributed to task 6 and 7 by using github and adding doxygen documentation to each class I made

Jayden:

- I was in charge of task 2. I used github to help coordinate all the work for task 2 with my group mates
- I was also responsible for task 5.1 and 5.2. We decided it would be smart to spread at task 5 due to the large amount of work it demanded.
- Like Yash I helped complete task 6 and 7, using github and including doxygen documentation to the classes I created
- Lastly I was in charge of Task 8, putting together a main and makefile and creating our demonstration route. I also made sure the pdf had all the required information and that our final practical adhered to all the requirements.

Simon:

- My job was to complete task 3, using github to share the code with my members.
- I, along with Jayden, tackled task 5. Taking responsibility over 5.3 and 5.4
- Finally, like my group members before me, I contributed to task 6 and 7.